

DESIGN AND IMPLEMENTATION OF LOW POWER FIFO BASED ROUTER

A PROJECT REPORT

Submitted by

YUVAN BALA P	61772231048
KRITHIKRAJ T	61772231203
SUBHIKSHA S	61772231506

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

GOVERNMENT COLLEGE OF ENGINEERING, SALEM-11

(An Autonomous Institution Affiliated to Anna University, Chennai, Accredited by
NAAC)

APRIL 2026

**GOVERNMENT COLLEGE OF ENGINEERING,
SALEM –11.**

(An Autonomous Institution Affiliated to Anna University, Chennai,
Accredited by NAAC)

BONAFIDE CERTIFICATE

Certified that this project report “**DESIGN AND IMPLEMENTATION
OF LOW POWER FIFO BASED ROUTER**” is the Bonafide work of

YUVAN BALA P 61772231048

KRITHIKRAJ T 61772231203

SUBHIKSHA S 61772231506

who carried out the project work under my supervision.

SIGNATURE

HEAD OF THE DEPARTMENT

Dr. D. MARY SUGANTHARATHNAM

Head of the Department

Department of Electronics and

Communication Engineering,

Government College of Engineering,

Salem – 636011

SIGNATURE

SUPERVISOR

Dr. K. JAYANTHI

Assistant Professor,

Department of Electronics and

Communication Engineering,

Government College of Engineering,

Salem – 636011

Submitted for University Project Viva-Voce held on:

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

At this delightful moment of having successfully completed our project, we convey our sincere thanks and gratitude to our beloved Principal **Dr. V. GEETHA**, for providing us with the great opportunity with all facilities required for the successful completion of our work.

We wish to express our gratitude and profound thanks to our Head of the Department **Dr. D. MARY SUGANTHARATHNAM**, Department of Electronics and Communication Engineering, Government College of Engineering, Salem, for forwarding us to do our project and offering adequate duration and constant encouragement in completing the project work.

We wish to record our deep sense of gratitude and profound thanks to our Project Supervisor **Dr. K. JAYANTHI**, Assistant Professor, Department of Electronics and Communication Engineering, Government College of Engineering, Salem, who has induced us to innovate this work with technological intents, inspiring guidance, keen interest, and constant encouragement throughout project work, to bring this report into fruition.

We would like to thank the Project Review Committee Members for their comments which helped to improve the overall quality of the project work. We also convey our sincere thanks to all the teaching faculty members and non-teaching staff members of our department, for their kind help and valuable support which facilitate us to complete the project work successfully. Finally, we wish to express our heartfelt thanks to our beloved parents and all our friends for the kindly help rendered by them

ABSTRACT

The rapid growth of digital communication systems and System-on-Chip (SoC) technologies has increased the demand for efficient, high-speed, and low-power data routing mechanisms. Routers play a vital role in directing data packets between different modules in such systems, and their performance significantly impacts the overall system efficiency. This project presents the design and implementation of a low power FIFO-based router using Verilog Hardware Description Language (HDL).

The proposed router architecture incorporates First-In-First-Out (FIFO) buffers to manage and regulate the flow of data packets. FIFO buffers ensure that data is transmitted in the same order in which it is received, thereby maintaining data integrity and preventing congestion. The router is designed with one input and multiple output ports, along with control logic that determines the appropriate path for each data packet based on the routing algorithm.

A key focus of this project is the reduction of power consumption without compromising performance. This is achieved by optimizing the FIFO operations and control logic, thereby minimizing switching activity and unnecessary data transitions. The design is modeled and implemented using Verilog HDL, which provides flexibility, scalability, and ease of simulation.

The functionality of the proposed router is verified through simulation using standard tools such as Xilinx Vivado. Simulation results demonstrate correct data transfer, efficient buffering, and improved power performance compared to conventional routing techniques. The proposed FIFO-based router is suitable for applications in Network-on-Chip (NoC), embedded systems, and other digital communication architectures where low power and reliable data transmission are critical.

In conclusion, this work successfully delivers a power-efficient and reliable routing solution using FIFO-based architecture.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
	LIST OF TABLES	vi
	LIST OF FIGURES	vi
1	INTRODUCTION	1
	1.1 Introduction	1
	1.2 Background	2
	1.3 Motivation	2
	1.4 Problem Statement	3
	1.5 Objectives of the Project	3
	1.6 Scope of the Project	4
2	LITERATURE SURVEY	5
	2.1 FIFO-Based Router Designs	5
	2.2 Low Power Router Designs	5
	2.3 Routing Algorithms	6
	2.4 Verilog-Based Implementations	6
	2.5 Optimized FIFO and Buffer Management	7
	2.6 Research Gap	7
3	SYSTEM ANALYSIS	8
	3.1 Existing System	8
	3.1.1 Architecture of Existing Routers	8
	3.1.2 Buffer Management	8
	3.1.3 Routing Mechanism	8

	3.1.4 Power Consumption Issues	8
	3.1.5 Limitations	8
	3.2 Proposed System	9
	3.2.1 FIFO-Based Architecture	9
	3.2.2 Efficient Buffer Management	9
	3.2.3 Control Logic Optimization	9
	3.2.4 Power Reduction	9
	3.2.5 Modular Verilog Design	9
	3.3 Comparison of Existing and Proposed System	10
4	SYSTEM DESIGN	11
	4.1 Overall Architecture of the FIFO-based router	11
	4.2 Block Diagram Description	12
	4.2.1 FSM Controller	12
	4.2.2 Register	12
	4.2.3 Synchronizer	13
	4.2.4 FIFO Buffers	14
	4.2.5 Output Logic	15
	4.3 Working of the System	15
	4.4 Power Optimization	16
	4.5 Advantages of Proposed System	16
5	IMPLEMENTATION	17
	5.1 Tools and Technologies Used	17
	5.1.1 Vivado Design Suite	17
	5.1.2 FPGA Board (Basys 3)	17
	5.2. Design Entry	18
	5.3 Simulation	18
	5.4 Synthesis and Implementation	19

	5.5 Hardware Implementation	20
	5.6 Testing and Verification	20
	5.7 Power Optimization	21
6	RESULTS AND VERIFICATION	22
	6.1 Simulation Results	22
	6.1.1 FSM Controller Simulation	22
	6.1.2 Register Simulation	22
	6.1.3 FIFO Simulation	23
	6.1.4 Synchronizer Simulation	24
	6.1.5 Top Module Simulation	25
	6.2 Functional Verification	25
	6.3 Design Visualization	26
	6.4 Resource Utilization	26
	6.5 Timing Analysis	27
	6.6 Power Analysis	28
	6.7 Hardware Verification	28
7	CONCLUSION AND FUTURE WORK	30
	7.1 Conclusion	30
	7.2 Future Work	30
	REFERENCES	32
	APPENDIX 1	33

LIST OF TABLES

TABLE	TITLE	PAGE NO.
3.1	Comparison of Existing and Proposed System	10
5.1	Software and Hardware Tools	17
5.2	Expected Result	21

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
4.1	Architecture of FIFO-Based Router	11
4.2	FSM Controller Module	12
4.3	Register Module	13
4.4	Synchronizer Module	14
4.5	FIFO Buffer Module	15
5.1	FPGA Board	18
5.2	Timing Diagram	19
5.3	Hardware Implementation	20
6.1	FSM Controller Waveform	22
6.2	Register Waveform	23
6.3	FIFO Waveform	24
6.4	Synchronizer Waveform	24
6.5	Complete Router Waveform	25
6.6	Gate-level Design	26
6.7	Resource Utilization	27
6.8	Timing Summary	27
6.9	Power Summary	28
6.10	FPGA Verification	29

CHAPTER 1

INTRODUCTION

1.1 Introduction

In recent years, the rapid advancement of digital systems and semiconductor technologies has led to the development of highly complex integrated circuits. Modern electronic systems such as System-on-Chip (SoC) and Network-on-Chip (NoC) architectures integrate multiple functional modules onto a single chip. These modules must communicate efficiently with each other to ensure proper system functionality. As a result, the design of efficient communication infrastructure has become a critical aspect of digital system design.

Routers are one of the most essential components in such communication systems. They are responsible for directing data packets from a source module to a destination module through appropriate paths. The performance of a router directly affects the overall efficiency, speed, and reliability of the system. Therefore, designing a router that is both high-performance and power-efficient is a major challenge in modern VLSI design.

With the continuous scaling of technology, power consumption has emerged as a significant concern. High power consumption not only reduces battery life in portable devices but also leads to excessive heat generation, which can affect system reliability and performance. Hence, there is a growing need to design low-power digital systems without compromising their performance.

This project focuses on the design and implementation of a low power FIFO-based router using Verilog Hardware Description Language (HDL). The proposed design aims to improve data handling efficiency while reducing power consumption, making it suitable for modern digital communication systems.

1.2 Background

In digital communication systems, data is transmitted in the form of packets between different modules. Efficient handling and routing of these data packets are essential to maintain system performance. One of the key challenges in such systems is managing the flow of data to avoid congestion, data loss, and delays. First-In-First-Out (FIFO) buffers are widely used in digital systems for temporary data storage. A FIFO buffer operates on a simple principle where the first data element entering the buffer is the first one to be removed. This ensures that data is processed in the correct order, maintaining data integrity and synchronization. FIFO buffers play a crucial role in router design by:

- Reducing data congestion
- Managing variable data rates
- Providing temporary storage for incoming packets

In addition, the use of Hardware Description Languages (HDLs) such as Verilog has revolutionized digital design. Verilog allows designers to describe hardware behavior at various levels of abstraction and simulate the design before physical implementation. This reduces design errors, saves time, and improves overall efficiency.

1.3 Motivation

The motivation behind this project arises from the increasing demand for energy-efficient and high-performance communication systems. Traditional router designs often focus on performance while neglecting power consumption. However, in modern applications such as mobile devices, embedded systems, and IoT devices, power efficiency is equally important.

Key motivating factors include:

- Need for low power consumption in VLSI systems
- Requirement for efficient data handling mechanisms
- Increasing complexity of on-chip communication systems
- Demand for scalable and reliable router architectures

By integrating FIFO buffers with optimized control logic, this project aims to address these challenges and provide an effective routing solution.

1.4 Problem Statement

Despite significant advancements in router design, several challenges still exist in achieving an optimal balance between performance and power consumption. Conventional routers often suffer from the following issues:

- High dynamic power consumption due to excessive switching activity
- Inefficient data buffering leading to congestion.
- Increased latency in packet transmission
- Lack of scalability for complex systems

These limitations highlight the need for a router design that can efficiently manage data flow while minimizing power usage.

The problem addressed in this project is to design a FIFO-based router that:

- Reduces power consumption
- Ensures efficient data transmission
- Maintains data integrity and reliability

1.5 Objectives of the Project

The primary objectives of this project are as follows:

- To design a low power router using FIFO buffering techniques
- To implement the router architecture using Verilog HDL
- To ensure efficient handling of data packets with minimal delay
- To reduce power consumption by optimizing switching activity
- To simulate and verify the functionality of the design
- To analyze the performance of the proposed system

1.6 Scope of the Project

This project focuses on the design and simulation of a FIFO-based router suitable for digital communication systems. The scope includes developing a modular router architecture, implementing FIFO buffers, and designing control logic for routing decisions.

The proposed system can be applied in:

- Network-on-Chip (NoC) architectures
- Embedded systems
- VLSI-based communication systems
- Real-time data processing applications

Although the current work is limited to simulation, the design can be extended for hardware implementation using FPGA platforms in future.

CHAPTER 2

LITERATURE SURVEY

2.1 FIFO-Based Router Designs

S. Murali, L. Benini, and G. De Micheli (2005) proposed a Network-on-Chip (NoC) architecture that uses FIFO buffers at the input ports of routers to handle incoming data packets. Their design ensures that packets are transmitted in the same order in which they arrive, thereby maintaining data integrity. The use of FIFO buffers helps in managing burst traffic and reducing packet loss, which improves overall throughput. However, the continuous read and write operations in FIFO buffers lead to increased power consumption, as switching activity is not minimized.

William J. Dally and Brian Towles (2001) introduced the fundamental concepts of interconnection networks and router architectures. Their work emphasized the importance of buffering techniques, including FIFO, in ensuring reliable data communication and avoiding congestion. Although their research provided a strong theoretical foundation for router design, it did not focus on power optimization, which has become an important factor in modern VLSI systems.

2.2 Low Power Router Designs

Luca Benini and Giovanni De Micheli (2002) presented techniques for designing low-power Network-on-Chip architectures. Their research focused on reducing dynamic power consumption by minimizing switching activity in digital circuits. They demonstrated that optimizing communication patterns and reducing unnecessary data transitions can significantly improve energy efficiency in routers.

H. Wang, L. Peh, and S. Malik (2010) proposed a low-power router design using clock gating techniques. In their approach, the clock signal is disabled for inactive

modules, thereby reducing unnecessary switching activity. Their results showed considerable power savings; however, the design required additional control circuitry, which increased system complexity and area.

J. Kim, D. Park, and T. Vijaykumar (2003) introduced dynamic voltage scaling (DVS) as a method to reduce power consumption in router architectures. By adjusting the supply voltage according to workload conditions, their design achieved significant energy savings. However, this technique required complex control mechanisms and additional hardware support, making the design more complicated.

2.3 Routing Algorithms

C. J. Glass and L. M. Ni (1992) proposed the XY routing algorithm, which is a deterministic routing technique widely used in mesh-based Network-on-Chip architectures. In this method, packets are first routed along the X-direction and then along the Y-direction. This ensures a simple and deadlock-free routing process. However, this approach lacks adaptability and may lead to congestion under high traffic conditions.

José Duato (1997) introduced adaptive routing techniques that dynamically select paths based on network conditions. His approach improves network performance by reducing congestion and increasing throughput. However, adaptive routing requires more complex control logic and additional hardware resources, which increases power consumption and design complexity.

2.4 Verilog-Based Router Implementations

A. K. Singh and R. Kumar (2012) designed and implemented a router using Verilog Hardware Description Language. Their design included FIFO buffers and routing control logic, following a modular approach. Simulation results showed correct data transmission and functional accuracy. However, their work mainly focused on functionality rather than optimizing power consumption.

Jan M. Rabaey, Anantha Chandrakasan, and Borivoje Nikolić (2003) emphasized the importance of Hardware Description Languages in digital system design. Their research demonstrated that Verilog allows efficient modeling, simulation, and verification of complex systems before hardware implementation. This approach reduces design errors and improves development efficiency.

2.5 Optimized FIFO and Buffer Management

Y. Liu, X. Zhang, and Z. Wang (2014) proposed an optimized FIFO design aimed at reducing switching activity within the buffer. Their design used efficient control logic and pointer management techniques to minimize unnecessary read and write operations. This resulted in reduced power consumption and improved system performance.

X. Zhang, Y. Chen, and L. Li (2016) introduced dynamic buffer management techniques that allocate buffer resources based on traffic conditions. Their approach improved data flow efficiency and reduced congestion in the network. However, the implementation required additional hardware resources, increasing the complexity of the design.

2.6 Research Gap

From the above research works, it is observed that FIFO-based routers improve data handling efficiency but often result in higher power consumption. Low-power techniques such as clock gating and voltage scaling effectively reduce energy usage but introduce additional complexity. Adaptive routing improves performance but increases hardware requirements and power consumption.

Therefore, there is a need for a simple, efficient, and low-power FIFO-based router design that can balance performance and energy consumption. This forms the basis for the proposed work.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

3.1.1 Architecture of Existing Routers

Traditional routers in Network-on-Chip (NoC) systems are designed with basic buffering units, routing logic, and switching mechanisms. Data packets arriving at input ports are temporarily stored in simple buffers or registers before being forwarded to output ports. These buffers are not always structured, leading to inefficient data handling during high traffic conditions.

3.1.2 Buffer Management

In existing systems, buffer management is not optimized. Data packets are stored without strict ordering mechanisms like FIFO, which may lead to out-of-order transmission or congestion. Continuous read and write operations increase switching activity, resulting in higher power consumption.

3.1.3 Routing Mechanism

Most traditional routers use deterministic routing algorithms such as XY routing. These methods follow a fixed path, ensuring simplicity and deadlock-free operation. However, they cannot adapt to varying traffic conditions, leading to uneven load distribution and congestion in certain paths.

3.1.4 Power Consumption Issues

Existing router designs consume high power due to continuous operation of all modules. Buffers, control logic, and routing units remain active even when not required. This leads to excessive switching activity, which increases dynamic power consumption and reduces overall efficiency.

3.1.5 Limitations

- High power consumption
- Inefficient buffer utilization

- Increased latency during heavy traffic
- Lack of flexibility in routing
- Poor scalability for large systems

3.2 Proposed System

3.2.1 FIFO-Based Architecture

The proposed system introduces FIFO (First-In-First-Out) buffers at the input ports of the router. This ensures that data packets are transmitted in the same order as they are received, maintaining data integrity and proper synchronization.

3.2.2 Efficient Buffer Management

FIFO buffers are designed with optimized read and write operations. Data is accessed only when required, reducing unnecessary switching activity. This improves buffer utilization and minimizes congestion.

3.2.3 Control Logic Optimization

The router uses simplified and efficient control logic to determine the routing path of data packets. The design avoids complex adaptive routing techniques, reducing hardware complexity while maintaining reliable performance.

3.2.4 Power Reduction Technique

Power consumption is minimized by controlling switching activity within FIFO buffers and routing logic. Modules operate only when necessary, reducing dynamic power usage and improving energy efficiency.

3.2.5 Modular Verilog Design

The system is implemented using Verilog HDL in a modular manner. Each component, such as FIFO, control unit, and routing logic, is designed separately. This improves scalability, testing, and ease of implementation.

3.3 Comparison of Existing and Proposed System

Existing Solution	Proposed Solution
Uses simple or unstructured buffers	Uses FIFO-based structured buffers
High power consumption	Low power consumption
Continuous switching activity	Reduced and controlled switching
Inefficient data handling	Efficient and ordered data transmission
Fixed or complex routing methods	Simplified and optimized routing
Higher latency under heavy traffic	Reduced congestion and latency
Less scalable design	Modular and scalable design

Table 3.1: Comparison of Existing and Proposed System

CHAPTER 4

SYSTEM DESIGN

4.1 Overall Architecture of the FIFO-Based Router

The proposed system is a low power FIFO-based router designed using Verilog HDL. The architecture is modular and consists of five major components: FSM Controller, Register, Synchronizer, and three FIFO buffers (FIFO 0, FIFO 1, FIFO 2) along with output logic.

The router is designed to receive data packets through a single input port and route them to one of the three output ports based on the destination address embedded within the packet. The system ensures ordered data transmission, efficient buffering, and reduced power consumption by minimizing unnecessary switching activity.

The operation of the system is fully controlled by the FSM controller, which coordinates all modules through control signals. The synchronizer ensures proper timing and signal alignment, while FIFO buffers manage temporary storage and maintain data order.

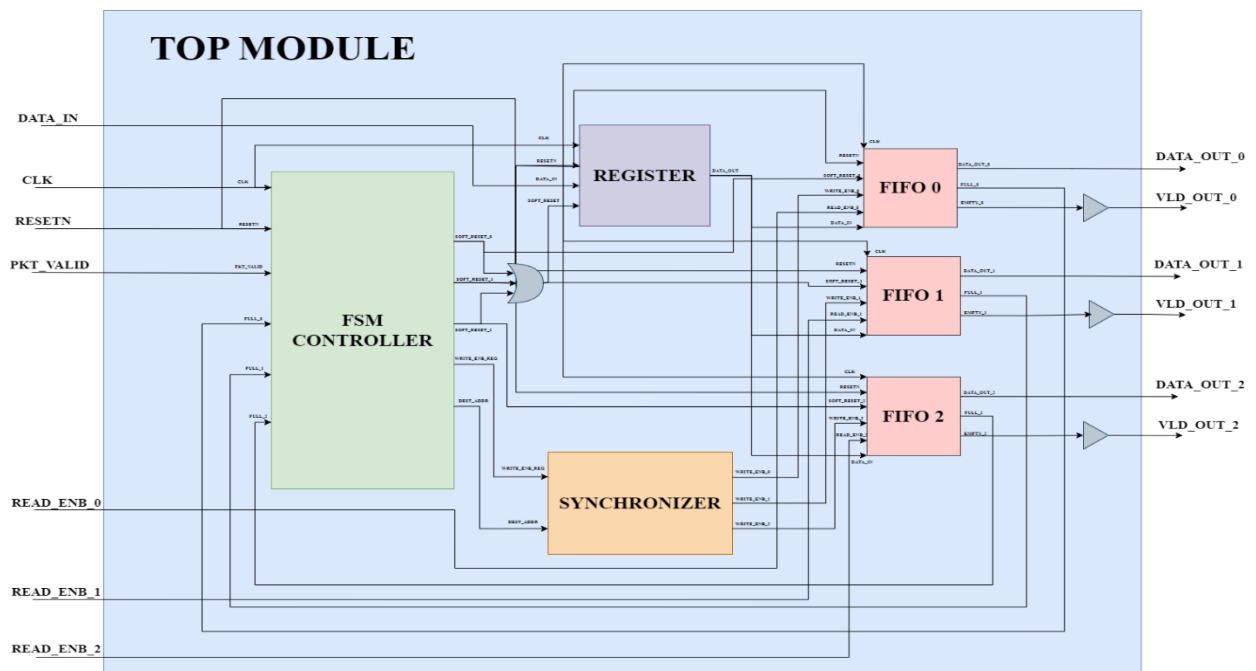


Figure 4.1: Architecture of FIFO based Router

4.2 Block Diagram Description

4.2.1 FSM Controller

The Finite State Machine (FSM) Controller is the central control unit of the router and plays a critical role in managing the entire data flow.

The FSM operates based on clock and reset signals and transitions through different states such as idle, data loading, routing decision, and data forwarding. It continuously monitors input signals like PKT_VALID, FIFO status signals (FULL, EMPTY), and read enable signals.

The primary functions of the FSM controller include:

- Controlling when data should be accepted from the input
- Generating control signals for the register and FIFO buffers
- Determining the destination FIFO based on packet information
- Managing write enable and read enable operations
- Preventing data overflow and underflow conditions

The FSM ensures that all operations occur in a synchronized manner, thereby maintaining system stability and avoiding conflicts between modules.

Router - Controller

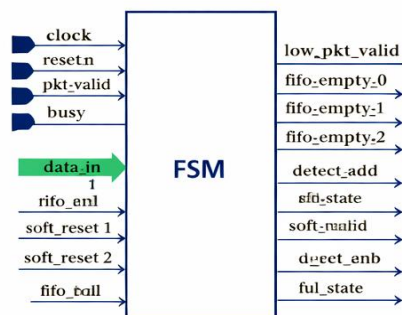


Figure 4.2: FSM Controller Module

4.2.2 Register

The **Register block** acts as an intermediate storage unit between the input and the FIFO buffers.

When a data packet arrives at the input, it is first stored in the register. This temporary storage allows the FSM controller to analyze the packet and determine

the appropriate routing path before forwarding it to the FIFO.

Functions of the register include:

- Holding incoming data packets temporarily
- Providing stable data to the FIFO buffers
- Synchronizing input data with system clock
- Supporting controlled data transfer based on FSM signals

The use of a register prevents direct data transfer from input to FIFO, which helps in reducing errors and improving timing control.

Router - Register

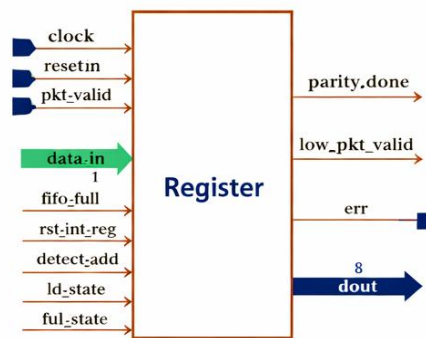


Figure 4.3: Register Module

4.2.3 Synchronizer

The Synchronizer block is responsible for coordinating control signals between different modules of the router.

In a complex digital system, different modules may operate with slight timing differences. The synchronizer ensures that all control signals are properly aligned with the system clock, avoiding timing mismatches and data corruption.

Key functions of the synchronizer include:

- Generating write enable signals for FIFO buffers (WRITE_ENB_0, WRITE_ENB_1, WRITE_ENB_2)
- Synchronizing control signals from FSM to FIFO
- Ensuring correct timing of read and write operations
- Preventing metastability issues

By properly managing timing, the synchronizer improves the reliability and efficiency of the system.

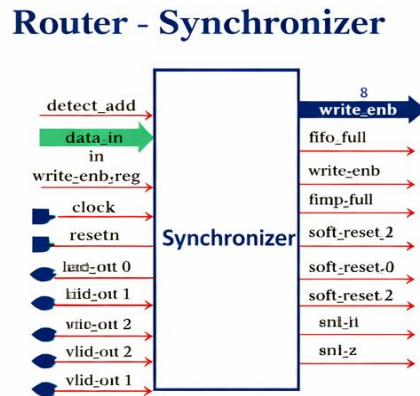


Figure 4.4: Synchronizer Module

4.2.4 FIFO Buffers (FIFO 0, FIFO 1, FIFO 2)

The router uses three FIFO buffers, each corresponding to a specific output port. FIFO (First-In-First-Out) is a memory structure where data is read in the same order as it is written. This ensures that packet sequence is maintained throughout the transmission.

Each FIFO performs the following operations:

- Stores incoming data packets
- Allows controlled read and write operations
- Maintains data order
- Generates status signals such as:
 - **FULL** → Indicates buffer is full
 - **EMPTY** → Indicates buffer is empty

The FIFO buffers play a crucial role in:

- Reducing data congestion
- Managing variable data rates
- Ensuring smooth data flow between modules

Each FIFO is connected to a specific output:

- FIFO 0 → DATA_OUT_0
- FIFO 1 → DATA_OUT_1
- FIFO 2 → DATA_OUT_2

Router - FIFO

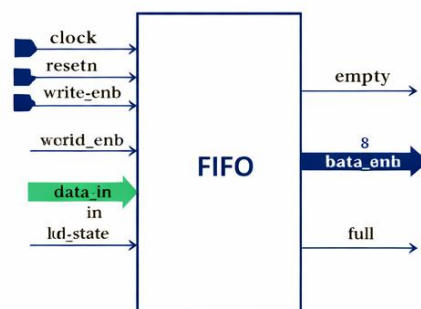


Figure 4.5: FIFO Buffer Module

4.2.5 Output Logic

The output logic is responsible for delivering the processed data to the external system.

Each output port consists of:

- **DATA_OUT signals** → Carry the actual data
- **VLD_OUT signals** → Indicate valid data availability

The valid signals ensure that the receiving module reads data only when it is ready, preventing incorrect data transfer.

4.3 Working of the System

The operation of the router follows a sequential and controlled process:

1. A data packet enters the system through DATA_IN
2. The PKT_VALID signal indicates that the incoming data is valid
3. The FSM controller detects valid data and initiates processing
4. The data is stored temporarily in the Register

5. The FSM analyzes the packet header to determine the destination output
6. Based on the decision, the synchronizer activates the corresponding FIFO write enable signal
7. The data is written into the selected FIFO (FIFO 0 / FIFO 1 / FIFO 2)
8. The FIFO stores data while maintaining order
9. When READ_ENB signals are activated, data is read from the FIFO
10. Data is sent to the corresponding output port
11. The VLD_OUT signal indicates that valid data is available at output

This process ensures smooth, ordered, and efficient data transfer.

4.4 Power Optimization

The proposed design focuses on reducing power consumption through:

- Controlled activation of FIFO read/write operations
- Minimization of unnecessary switching activity
- Efficient use of control signals
- Avoidance of continuous module operation

By ensuring that modules operate only when required, the system significantly reduces dynamic power consumption compared to traditional designs.

4.5 Advantages of the Proposed Architecture

- Maintains proper data order using FIFO
- Reduces congestion and packet loss
- Minimizes power consumption
- Provides modular and scalable design
- Improves overall system efficiency and reliability

CHAPTER 5

IMPLEMENTATION

5.1 Tools and Technologies Used

Category	Tool/Device	Description
Design Tool	Vivado Design Suite	Used for design, simulation, synthesis
Language	Verilog HDL	Hardware description language
Hardware	Basys 3 FPGA (XC7A35T-CPG236-1)	Used for implementation
Simulator	Vivado Simulator	Used for waveform verification

Table 5.1: Software and Hardware Tools

5.1.1 Vivado Design Suite

The design and simulation of the FIFO-based router are carried out using Xilinx Vivado Design Suite. Vivado provides an integrated environment for writing, synthesizing, simulating, and implementing Verilog-based digital designs.

It supports:

- RTL design entry using Verilog HDL
- Functional and timing simulation
- Synthesis and implementation
- Bitstream generation for FPGA

Vivado is widely used in VLSI design due to its powerful debugging and optimization features.

5.1.2 FPGA Board (Basys 3 – XC7A35T-CPG236-1)

The hardware implementation is carried out using the Basys 3 FPGA board, which is based on the Artix-7 FPGA.

Key features:

- FPGA device: XC7A35T-CPG236-1
- 33,280 logic cells

- On-board clock source
- LEDs, switches, and I/O pins
- USB programming support

The FPGA allows real-time testing of the designed router and verifies its functionality in hardware.

Basys3: Pmod Pin-Out Diagram

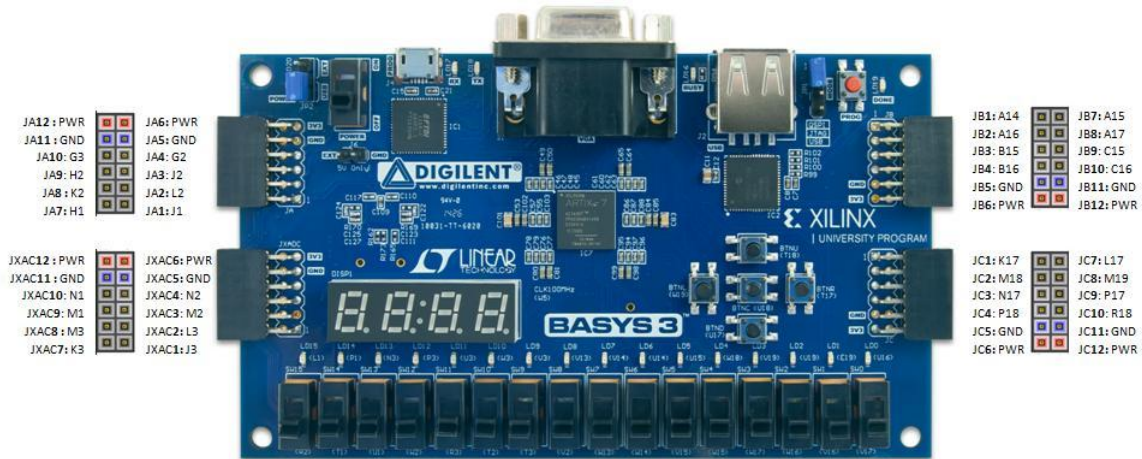


Figure 5.1: FPGA Board

5.2 Design Entry

The router is designed using Verilog HDL in a modular manner. Each functional block is implemented as a separate module.

Modules implemented:

- FSM Controller
- Register
- Synchronizer
- FIFO (three instances)
- Top Module

The top module integrates all submodules and defines input/output connections.

5.3 Simulation

Simulation is carried out using the Vivado simulator to verify the functionality of the design before hardware implementation. Input signals such as clock, reset,

and valid data are applied, and the corresponding outputs are observed in waveform format.

The simulation results confirm that the router correctly processes input data and routes it to the appropriate output ports. FIFO buffers maintain proper data order, and control signals generated by the FSM operate correctly. The output valid signals indicate that the data is transmitted successfully. Simulation ensures that the design is free from logical errors and ready for hardware implementation.

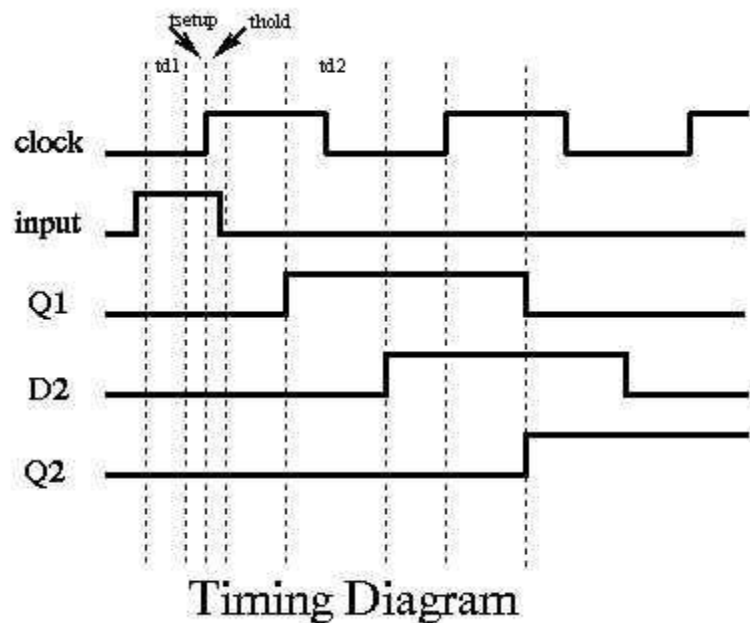


Figure 5.2: Timing Diagram

5.4 Synthesis and Implementation

During synthesis, the Verilog code is converted into a gate-level representation and optimized for FPGA implementation. The design is mapped onto available hardware resources, including lookup tables, flip-flops, and memory blocks. The implementation stage performs placement and routing, ensuring that the design meets timing requirements and operates efficiently.

The synthesis and implementation results show that the design utilizes FPGA resources effectively while maintaining low power consumption. The optimized design ensures reliable operation without exceeding hardware constraints.

5.5 Hardware Implementation

After successful synthesis and implementation, the bitstream file is generated and loaded onto the Basys 3 FPGA board. The board is connected to the system using a USB interface, and the design is programmed using Vivado.

Input signals are provided through onboard switches, and the output is observed using LEDs. This allows real-time verification of the router functionality. The hardware testing confirms that the system operates correctly and produces results consistent with simulation.

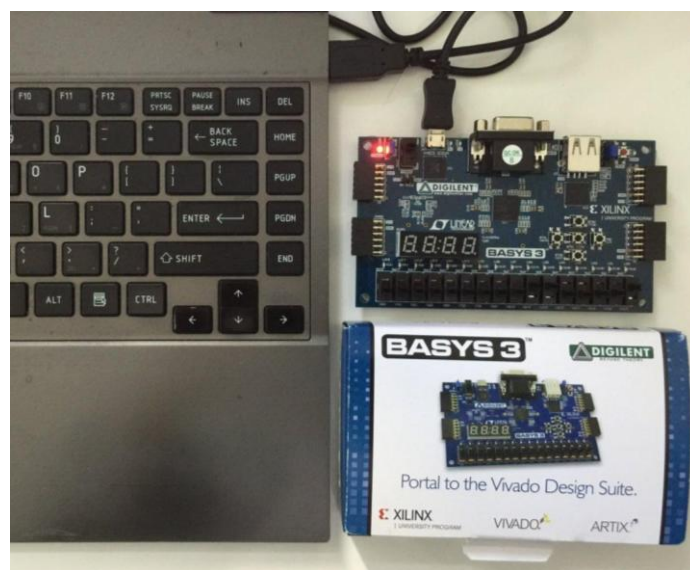


Figure 5.3: Hardware Implementation

5.6 Testing and Verification

The system is tested under different input conditions to ensure correct operation. Various test cases are applied, including valid data input, FIFO full condition, FIFO empty condition, and continuous data transmission. In all cases, the router performs as expected, correctly routing data and maintaining proper sequence.

The FIFO buffers effectively handle data flow, and the FSM controller ensures proper coordination of operations. The results confirm that the design is reliable and suitable for practical applications.

Test Case	Expected Output
Valid Packet	Routed correctly
FIFO Full	No write operation
FIFO Empty	No read operation
Multiple Inputs	Correct sequencing

Table 5.2: Expected Result

5.7 Power Optimization

The proposed design achieves low power consumption by minimizing switching activity within the system. FIFO operations are controlled efficiently, ensuring that read and write operations occur only when necessary. The FSM controller manages the activation of different modules, preventing unnecessary activity.

By optimizing data flow and reducing redundant operations, the design achieves improved energy efficiency compared to conventional router architectures. This makes the system suitable for low-power digital applications.

CHAPTER 6

RESULT

6.1 Simulation Results

6.1.1 FSM Controller Simulation

The FSM waveform shows the operation of control signals such as clk, resetn, pkt_valid, and dest_addr. Initially, when reset is active, the FSM remains in the idle state. When pkt_valid becomes high, the FSM detects valid data and transitions to the active state.

The destination address (dest_addr) changes according to the input packet, and corresponding write enable signals are generated for the selected FIFO. Only one FIFO is enabled at a time, ensuring correct routing. The FSM also controls data flow and prevents invalid operations such as writing when FIFO is full.

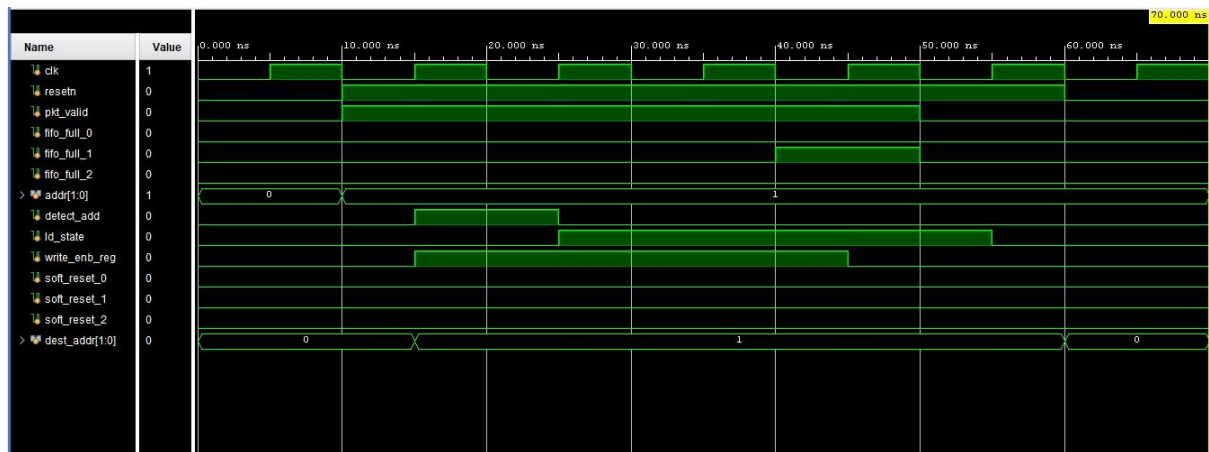


Figure 6.1: FSM Controller Waveform

Thus, the waveform confirms correct state transitions and proper control signal generation.

6.1.2 Register Simulation

The register waveform shows the behavior of signals such as clk, resetn, data_in, and data_out. Initially, when reset is active, the register output is cleared. When the clock signal is active and reset is deactivated, the input data is stored in the

register on the rising edge of the clock.

The waveform shows that data_out follows data_in with a clock delay, confirming correct data storage and transfer. The register ensures stable and synchronized data flow between modules.

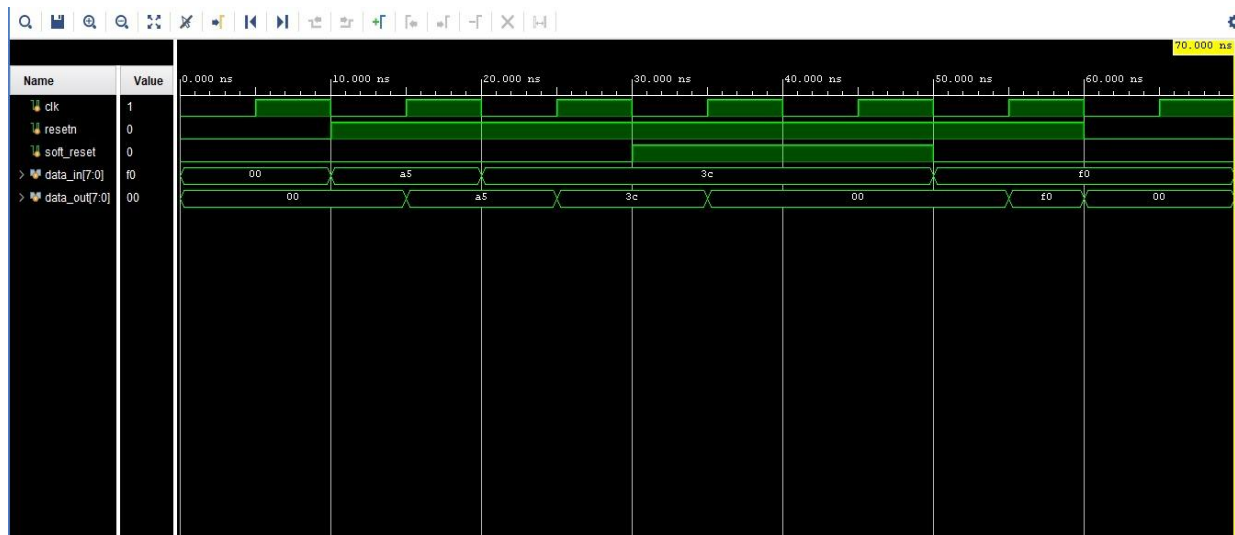


Figure 6.2: Register Waveform

Thus, the simulation verifies proper operation of the register module.

6.1.3 FIFO Simulation

The FIFO waveform shows the operation of signals such as clk, resetn, write_enb, read_enb, data_in, and data_out. When the write enable signal is active, data is written into the FIFO, and when the read enable signal is active, data is read from the FIFO.

The waveform confirms that the data is output in the same order as it is input, demonstrating correct First-In-First-Out behavior. The full and empty signals are generated appropriately to prevent overflow and underflow conditions.

Thus, the simulation verifies correct data storage, retrieval, and control operation of the FIFO module.

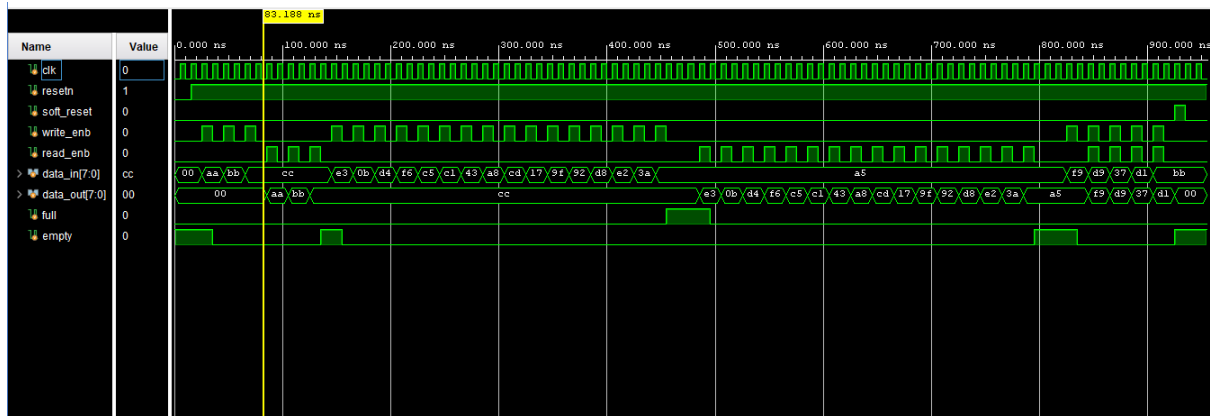


Figure 6.3: FIFO Waveform

6.1.4 Synchronizer Simulation

The synchronizer waveform shows the generation of control signals such as write enable signals based on the destination address and input conditions. The signals are aligned with the clock to ensure proper timing.

The waveform confirms that only the required write enable signal is activated for the selected FIFO, while others remain inactive. This ensures correct routing and prevents data conflicts.

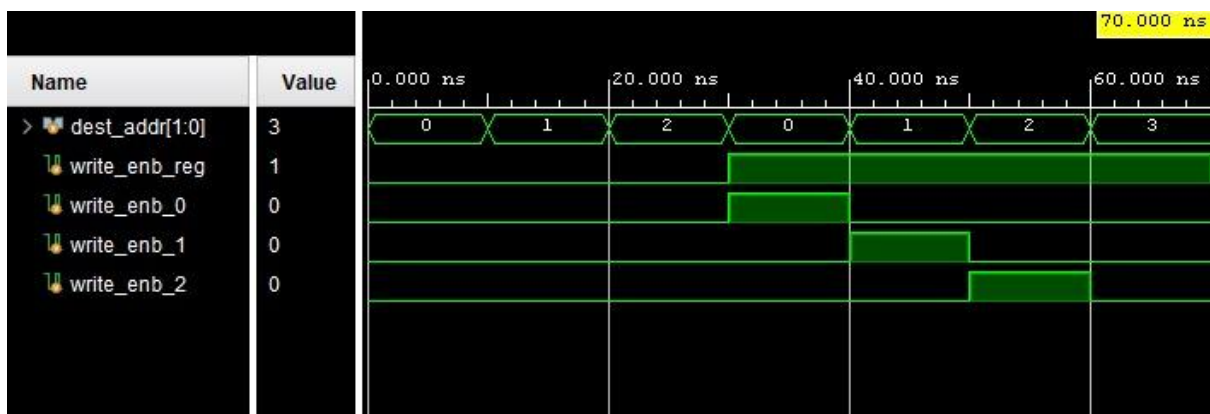


Figure 6.4: Synchronizer Waveform

Thus, the simulation verifies proper synchronization of control signals and reliable coordination between modules.

6.1.5 Top Module (Complete Router) Simulation

The top module waveform represents the complete operation of the FIFO-based router by integrating all modules such as FSM, register, synchronizer, and FIFO buffers. The waveform shows signals like `data_in`, `pkt_valid`, `data_out_0`, `data_out_1`, `data_out_2`, and valid outputs.

When valid input data is applied, it is processed through the register and controlled by the FSM. Based on the destination address, the synchronizer activates the appropriate FIFO, and the data is routed to the corresponding output. The waveform confirms that data is correctly transferred to the selected output port with proper valid signals, maintaining correct order and synchronization.



Figure 6.5: Complete Router Waveform

Thus, the simulation verifies the overall functionality and correct integration of the complete router system.

6.2 Functional Verification

Functional verification is performed to ensure that the router operates correctly under different input conditions. Various test cases such as valid packet input, FIFO full condition, FIFO empty condition, and continuous data transmission are applied to the system.

The results show that the router correctly routes data to the appropriate output based on the destination address. The FIFO buffers maintain proper data order, and the control logic prevents invalid operations such as writing when the FIFO

is full or reading when it is empty.

Thus, functional verification confirms that the system behaves as expected and performs reliable data transmission under all conditions.

6.3 Design Visualization

The design visualization is obtained using Vivado, which provides different views such as RTL schematic, synthesized schematic, and implementation layout. The RTL view shows the logical structure of the design, including modules like FSM, register, synchronizer, and FIFO interconnected together.

The synthesized schematic represents the gate-level implementation of the design, showing how the logic is mapped into basic digital components. The implementation view displays the physical placement and routing of the design on the FPGA, indicating how resources such as LUTs and flip-flops are utilized. These visualizations confirm that the design is correctly structured, efficiently implemented, and properly mapped onto the FPGA hardware.

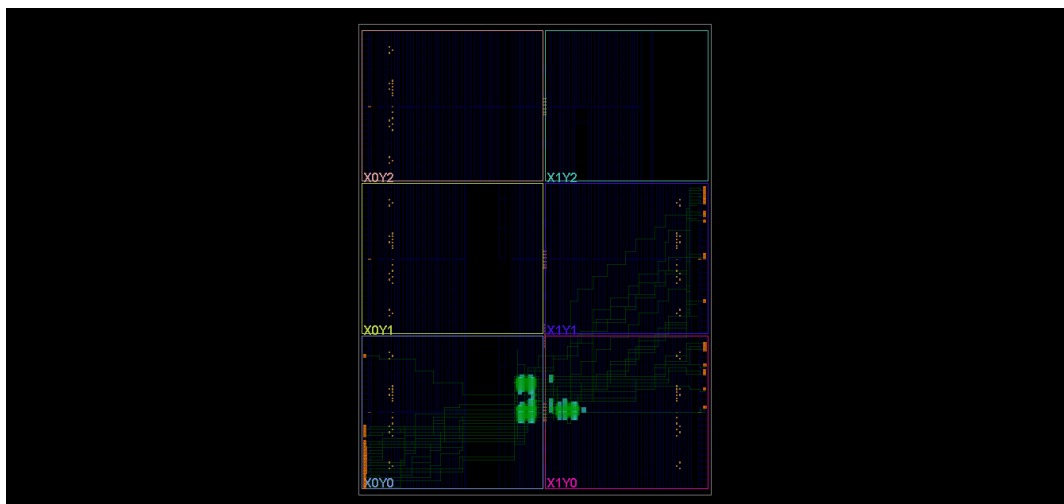


Figure 6.6: Gate-level Design

6.4 Resource Utilization

The resource utilization report is generated after synthesis and implementation in Vivado. It shows how efficiently the design uses FPGA resources such as lookup tables (LUTs), flip-flops, slices, and I/O pins.

The results indicate that the router design utilizes a moderate number of resources, with efficient usage of LUTs and registers. The FIFO buffers are implemented using available memory resources, and the overall design fits well within the FPGA capacity.

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
router_1x3	163	460	48	24	132	163	41	1
fifo0 (router_fifo)	52	149	16	8	46	52	0	0
fifo1 (router_fifo_0)	51	149	16	8	39	51	0	0
fifo2 (router_fifo_1)	51	149	16	8	45	51	0	0

Figure 6.7: Resource Utilization

Thus, the resource utilization confirms that the proposed design is efficient, scalable, and suitable for implementation on the selected FPGA device.

6.5 Timing Analysis

The timing analysis report is generated after implementation to verify whether the design meets the required timing constraints. It includes parameters such as worst negative slack (WNS), hold slack, and timing violations.

The results show that the design has positive slack values, indicating that all signals meet the required timing constraints without any violations. This ensures that data is transferred correctly between modules without delay or timing errors.

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.155 ns	Worst Hold Slack (WHS): 0.161 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 899	Total Number of Endpoints: 899	Total Number of Endpoints: 461
All user specified timing constraints are met.		

Figure 6.8: Timing Summary

Thus, the timing analysis confirms that the router operates reliably at the desired clock frequency and satisfies all timing requirements.

6.6 Power Analysis

The power analysis report is generated using Vivado to evaluate the power consumption of the design. It includes parameters such as dynamic power, static power, and total power consumption.

The results show that the dynamic power consumption is very low due to reduced switching activity, while the static power remains minimal. The overall power consumption of the design is optimized, demonstrating the effectiveness of the low-power FIFO-based architecture.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	0.069 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	25.3°C
Thermal Margin:	59.7°C (11.9 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium
Launch Power Constraint Advisor to find and fix invalid switching activity	

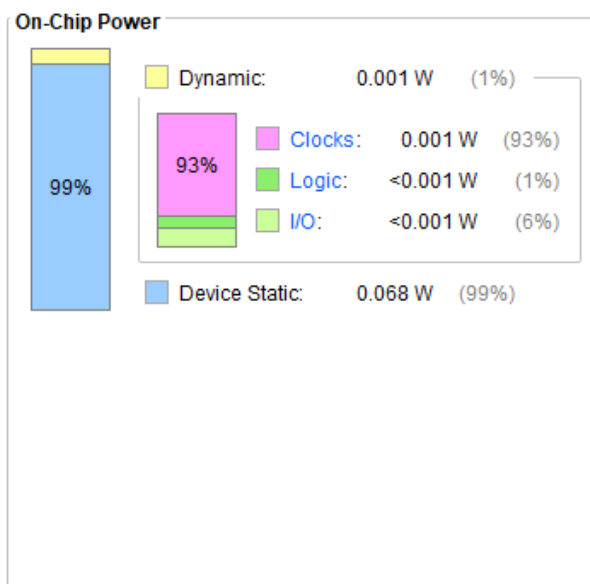


Figure 6.9: Power Summary

Thus, the power analysis confirms that the proposed router design is energy efficient and suitable for low-power applications.

6.7 Hardware Verification

The hardware verification is carried out by implementing the design on the Basys 3 FPGA board. The generated bitstream is loaded onto the FPGA, and input signals are applied using onboard switches.

The output is observed through LEDs, which indicate the correct routing of data

based on the destination address. The results observed on the hardware match the simulation results, confirming that the router operates correctly in real-time conditions.

Thus, hardware verification validates the practical implementation and reliability of the proposed design.

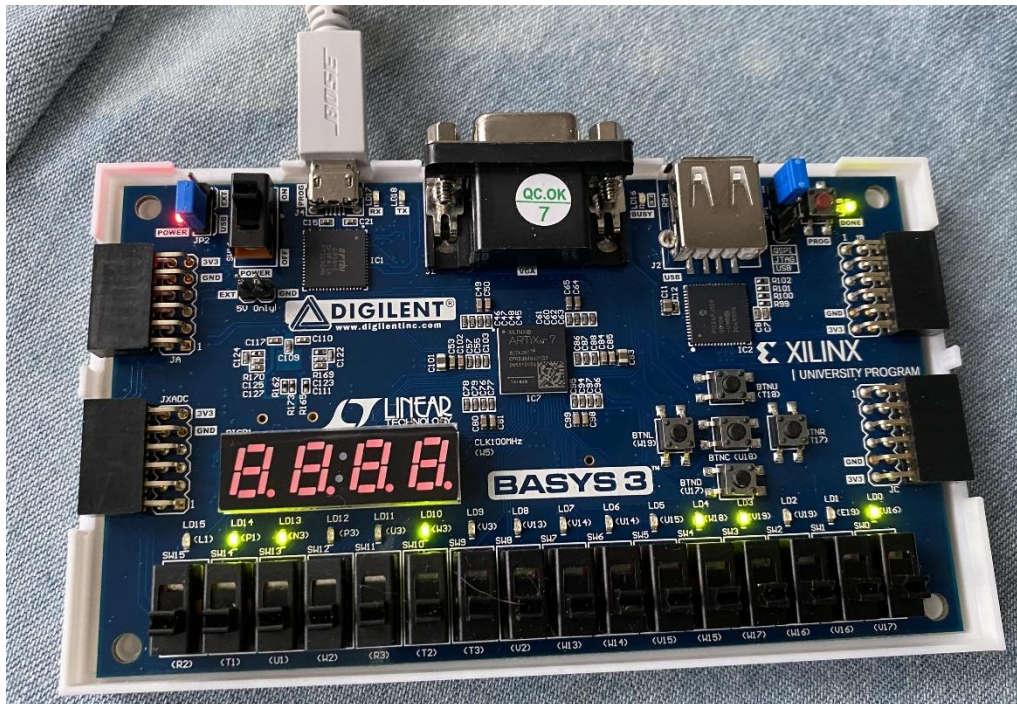


Figure 6.10: FPGA Verification

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project titled “Design and Implementation of Low Power FIFO-Based Router Using Verilog” has been successfully designed, simulated, and implemented. The proposed router utilizes FIFO buffers to ensure efficient data handling and maintains the correct order of packet transmission.

The system was developed using Verilog HDL and implemented using the Vivado Design Suite. Functional simulation results verified the correct operation of individual modules such as FSM controller, register, synchronizer, and FIFO buffers. The complete system simulation confirmed proper data routing and control signal generation.

The design was further implemented on the Basys 3 FPGA board, and hardware verification results matched the simulation outputs. Resource utilization reports showed efficient use of FPGA resources, while timing analysis confirmed that all timing constraints were satisfied. Power analysis demonstrated reduced power consumption due to controlled switching activity and optimized design.

Overall, the proposed FIFO-based router achieves reliable data transmission, reduced congestion, and improved energy efficiency, making it suitable for modern VLSI and Network-on-Chip applications.

7.2 Future Work

The proposed design can be further enhanced in several ways to improve performance and extend its applications.

- The router can be extended to support a larger number of input and output ports for complex systems.
- Advanced routing algorithms such as adaptive routing can be implemented

to improve performance under varying traffic conditions.

- The design can be optimized further to reduce area and power consumption.
- Error detection and correction mechanisms can be added to improve reliability.
- The system can be integrated into large-scale Network-on-Chip architectures for real-time applications.

These enhancements will make the router more efficient, scalable, and suitable for advanced digital communication systems.

REFERENCES

1. Benini, L. and De Micheli, G. (2002). Networks on chips: A new SoC paradigm. *Computer*, 35(1), pp.70–78.
2. Dally, W.J. and Towles, B. (2001). Route packets, not wires: On-chip interconnection networks. In *Proceedings of the Design Automation Conference*, pp.684–689.
3. Duato, J. (1997). A new theory of deadlock-free adaptive routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems*, 4(12), pp.1320–1331.
4. Kim, J., Dally, W.J. and Abts, D. (2003). Adaptive routing techniques in interconnection networks. In *Proceedings of the International Symposium on Computer Architecture*, pp.123–134.
5. Liu, Y., Zhang, X. and Wang, Z. (2014). Design of low-power FIFO buffer for network-on-chip applications. *International Journal of Electronics and Communication Engineering*, 8(4), pp.210–215.
6. Murali, S., Benini, L. and De Micheli, G. (2005). Designing application-specific networks on chips with floorplan information. In *Proceedings of the International Conference on Computer Design*, pp.355–362.
7. Rabaey, J.M., Chandrakasan, A. and Nikolić, B. (2003). *Digital Integrated Circuits: A Design Perspective*. 2nd ed. New Jersey: Prentice Hall.
8. Singh, A.K. and Kumar, R. (2012). Design and implementation of router using Verilog HDL. *International Journal of Engineering Research and Applications*, 2(5), pp.150–155.
9. Wang, H., Peh, L.S. and Malik, S. (2010). Power-driven design of router microarchitectures in network-on-chip. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, pp.105–116.

APPENDIX 1

This appendix contains the Verilog HDL code used for the design and implementation of the low power FIFO-based router. The design is implemented in a modular manner, where each functional block such as FSM controller, register, synchronizer, FIFO, and top module is coded separately.

A.1 Synchronizer

```
module router_sync (  
    input [1:0] dest_addr,  
    input write_enb_reg,  
    output reg write_enb_0,  
    output reg write_enb_1,  
    output reg write_enb_2  
);  
    always @(*) begin  
        write_enb_0 = 0;  
        write_enb_1 = 0;  
        write_enb_2 = 0;  
        if (write_enb_reg) begin  
            case(dest_addr)  
                2'b00: write_enb_0 = 1;  
                2'b01: write_enb_1 = 1;  
                2'b10: write_enb_2 = 1;  
                default: begin  
                    write_enb_0 = 0;  
                    write_enb_1 = 0;  
                    write_enb_2 = 0;  
                end  
            end  
        end  
    end
```

```

        end
    endcase
end
end
endmodule

```

A.2 FIFO Module

```

module router_fifo (
    input clk,
    input resetn,
    input soft_reset,
    input write_enb,
    input read_enb,
    input [7:0] data_in,
    output reg [7:0] data_out,
    output reg full,
    output reg empty
);
    reg [7:0] mem [0:15];
    reg [3:0] wr_ptr, rd_ptr;
    reg [4:0] count;
    integer i;
    always @(posedge clk or negedge resetn) begin
        if (!resetn) begin
            wr_ptr <= 0;
            rd_ptr <= 0;
            count <= 0;
            data_out <= 0;
            for (i = 0; i < 16; i = i + 1)
                mem[i] <= 0;

```

```

end else if (soft_reset) begin
    wr_ptr <= 0;
    rd_ptr <= 0;
    count <= 0;
    data_out <= 0;
end else begin
    if (write_enb && !full) begin
        mem[wr_ptr] <= data_in;
        wr_ptr <= wr_ptr + 1;
        count <= count + 1;
    end
    if (read_enb && !empty) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        count <= count - 1;
    end
end
end
always @(*) begin
    full = (count == 16);
    empty = (count == 0);
end
endmodule

```

A.3 FSM Controller

```

module router_fsm (
    input clk,
    input resetn,
    input pkt_valid,
    input fifo_full_0,

```

```

input fifo_full_1,
input fifo_full_2,
input [1:0] addr,
output reg detect_add,
output reg ld_state,
output reg write_enb_reg,
output reg soft_reset_0,
output reg soft_reset_1,
output reg soft_reset_2,
output reg [1:0] dest_addr
);
parameter IDLE  = 2'b00;
parameter DETECT = 2'b01;
parameter LOAD  = 2'b10;
reg [1:0] state, next_state;
always @(posedge clk or negedge resetn) begin
    if (!resetn)
        state <= IDLE;
    else
        state <= next_state;
end
always @(*) begin
    case(state)
        IDLE: begin
            if (pkt_valid)
                next_state = DETECT;
            else
                next_state = IDLE;
        end
    end
end

```

```

    DETECT: begin
        next_state = LOAD;
    end
    LOAD: begin
        if (!pkt_valid)
            next_state = IDLE;
        else
            next_state = LOAD;
        end
        default: next_state = IDLE;
    endcase
end

always @(posedge clk or negedge resetn) begin
    if (!resetn) begin
        detect_add    <= 0;
        ld_state      <= 0;
        write_enb_reg <= 0;
        dest_addr     <= 0;
        soft_reset_0  <= 0;
        soft_reset_1  <= 0;
        soft_reset_2  <= 0;
    end
    else begin
        detect_add    <= 0;
        ld_state      <= 0;
        write_enb_reg <= 0;
        soft_reset_0  <= 0;
        soft_reset_1  <= 0;
        soft_reset_2  <= 0;
    end
end

```

```

case(state)
  IDLE: begin
    if (pkt_valid) begin
      detect_add <= 1;
      write_enb_reg <=1;
      dest_addr <= addr;
    end
  end
  DETECT: begin
    ld_state    <= 1;
    write_enb_reg <= 1;
    dest_addr    <= addr;
  end
  LOAD: begin
    ld_state <= pkt_valid;
    case(dest_addr)
      2'b00: write_enb_reg <= pkt_valid && !fifo_full_0;
      2'b01: write_enb_reg <= pkt_valid && !fifo_full_1;
      2'b10: write_enb_reg <= pkt_valid && !fifo_full_2;
      default: write_enb_reg <= 0;
    endcase
  end
endcase
end
end
endmodule

```

A.4 Register Module

```

module router_reg (
  input clk,

```

```

    input resetn,
    input [7:0] data_in,
    input soft_reset,
    output reg [7:0] data_out
);
    always @(posedge clk or negedge resetn) begin
        if (!resetn || soft_reset)
            data_out <= 0;
        else
            data_out <= data_in;
    end
endmodule

```

A.5 Top Module

```

module router_1x3 (
    input clk,
    input resetn,
    input [7:0] data_in,
    input pkt_valid,
    input read_enb_0,
    input read_enb_1,
    input read_enb_2,
    output [7:0] data_out_0,
    output [7:0] data_out_1,
    output [7:0] data_out_2,
    output vld_out_0,
    output vld_out_1,
    output vld_out_2
);

```

```

wire [7:0] data_reg;
wire [1:0] dest_addr;
wire [1:0] addr;
wire write_enb_reg;
wire write_enb_0, write_enb_1, write_enb_2;
wire empty_0, empty_1, empty_2;
wire full_0, full_1, full_2;
wire soft_reset_0, soft_reset_1, soft_reset_2;
assign addr = data_in[1:0];
router_fsm fsm (
    .clk(clk),
    .resetn(resetn),
    .pkt_valid(pkt_valid),
    .fifo_full_0(full_0),
    .fifo_full_1(full_1),
    .fifo_full_2(full_2),
    .dest_addr(dest_addr),
    .addr(addr),
    .write_enb_reg(write_enb_reg),
    .soft_reset_0(soft_reset_0),
    .soft_reset_1(soft_reset_1),
    .soft_reset_2(soft_reset_2)
);
router_reg reg_block (
    .clk(clk),
    .resetn(resetn),
    .data_in(data_in),
    .soft_reset(soft_reset_0 | soft_reset_1 | soft_reset_2),
    .data_out(data_reg)

```



```

);
router_sync sync (
    .dest_addr(dest_addr),
    .write_enb_reg(write_enb_reg),
    .write_enb_0(write_enb_0),
    .write_enb_1(write_enb_1),
    .write_enb_2(write_enb_2)
);

router_fifo fifo0 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_0),
    .write_enb(write_enb_0),
    .read_enb(read_enb_0),
    .data_in(data_reg),
    .data_out(data_out_0),
    .full(full_0),
    .empty(empty_0)
);

router_fifo fifo1 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_1),
    .write_enb(write_enb_1),
    .read_enb(read_enb_1),
    .data_in(data_reg),
    .data_out(data_out_1),
    .full(full_1),
    .empty(empty_1)
);

```

```

);
router_fifo fifo2 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_2),
    .write_enb(write_enb_2),
    .read_enb(read_enb_2),
    .data_in(data_reg),
    .data_out(data_out_2),
    .full(full_2),
    .empty(empty_2)
);
assign vld_out_0 = !empty_0;
assign vld_out_1 = !empty_1;
assign vld_out_2 = !empty_2;
endmodule

```